

Guidelines for Writing Effective Use Cases

Guideline: Write in an Essential UI-Free Style

Key Idea

Write use cases in an **essential style**—free from user interface or technology details. Focus on user intentions and system responsibilities rather than how the user interacts with a GUI.

Example Scenario

During a requirements workshop, a cashier states their goal as “log in.” This describes a *mechanism* (GUI, password, etc.)—not the true goal. By asking “What is the goal of that goal?”, the analyst finds a higher-level, mechanism-independent goal:

- **Essential Goal:** Identify myself and get authenticated.
- **Higher Goal:** Prevent theft or misuse of the system.

Benefits of Essential Style

- Keeps focus on intent, not implementation.
- Encourages discovery of better solutions (e.g., biometric authentication instead of passwords).
- Avoids premature design decisions.

Essential vs. Concrete Style

- **Essential Style:** Focuses on user goals and system responsibilities. Avoids UI elements (e.g., screens, buttons).
- **Concrete Style:** Embeds UI decisions such as window layouts, dialogs, or widget interactions.

Guideline: Use essential style during early *requirements analysis*. Reserve concrete style for later stages—such as detailed GUI design.

Guideline: Write Terse Use Cases

Rule of Thumb

Be concise. Remove filler or redundant words.

Example:

- Use “System authenticates user” instead of “The system authenticates the user.”

Terse writing improves readability and helps maintain focus on key requirements.

Guideline: Write Black-Box Use Cases

Concept

A **black-box use case** describes what the system must do, not how it does it.

- Focus on **external behavior** visible to actors.
- Avoid internal design, data structures, or implementation details.

Why it matters: It separates *requirements analysis* (what) from *design* (how). This ensures clarity and flexibility during later system development.

Guideline: Take an Actor and Actor-Goal Perspective

Definition (Jacobson)

A use case is a set of use-case instances where each instance is a sequence of actions a system performs that yields an observable result of value to a particular actor.

Implications

- Focus requirements on **actors’ goals and valuable outcomes**.
- Ask: “What result provides value to the actor?”

Importance

Focusing on actor goals ensures that:

- Requirements reflect real user needs.
- Projects deliver observable value rather than mere features.